

Hands-On: Understanding Lifecycle Methods in React

Requirements

- Basic knowledge of React
- Text editor and web browser
- Node.js and npm installed on your computer

Instructions

1. Create a new React project using `create-react-app``.

```
npx create-react-app lifecycle-lab  
cd lifecycle-lab
```

2. Open the project folder in your text editor.
3. In the `src`` folder, create a new file named `LifecycleComponent.js``.

```
import React, { Component } from "react";  
  
class LifecycleComponent extends Component {  
  constructor(props) {  
    super(props);  
    this.state = {  
      count: 0,  
    };  
    console.log("Constructor");  
  }  
  
  componentDidMount() {  
    console.log("ComponentDidMount");  
  }  
  
  componentDidUpdate() {  
    console.log("ComponentDidUpdate");  
  }  
  
  componentWillUnmount() {  
    console.log("ComponentWillUnmount");  
  }  
  
  shouldComponentUpdate(nextProps, nextState) {  
    if (nextState.count !== this.state.count) {  
      console.log("ShouldComponentUpdate: Yes");  
      return true;  
    } else {  
      console.log("ShouldComponentUpdate: No");  
      return false;  
    }  
  }  
}
```

```

render() {
  console.log("Render");
  return (
    <div>
      <h1>Count: {this.state.count}</h1>
      <button onClick={() => this.setState({ count: this.state.count + 1 })}>
        Increase Count
      </button>
    </div>
  );
}
}

```

```
export default LifecycleComponent;
```

4. In the `App.js` file, import the `LifecycleComponent` and render it in the `render()` method.

```

import React from "react";
import LifecycleComponent from "../LifecycleComponent";

```

```

function App() {
  return (
    <div className="App">
      <LifecycleComponent />
    </div>
  );
}

```

```
export default App;
```

5. Start the development server.

```
npm start
```

6. Open your browser and go to `http://localhost:3000/`.
7. Open the console in your browser's developer tools. You should see the following logs in the console:

```

Constructor
Render
ComponentDidMount

```

This means that the `LifecycleComponent` is mounted, rendered, and its `componentDidMount()` method is called.

8. Click the "Increase Count" button a few times. You should see the following logs in the console:

```

ShouldComponentUpdate: Yes
Render

```

ComponentDidUpdate

This means that the `shouldComponentUpdate()` method is called before the component is updated, and then the `Render` and `componentDidUpdate()` methods are called after the component is updated.

9. Remove the `LifecycleComponent` from the `App.js` file.

```
import React from "react";
import "./App.css";

function App() {
  return <div className="App"></div>;
}

export default App;
```

10. Save the file and refresh the browser. You should see the following log in the console:

ComponentWillUnmount

This means that the `componentWillUnmount()` method is called when the component is unmounted from the DOM.

That's it! You've successfully completed the hands-on lab for lifecycle methods such as `componentDidMount`, `componentDidUpdate`, `componentWillUnmount`, and `shouldComponentUpdate` in React. You can now use these concepts to build more complex React applications with components that respond to different stages of their lifecycle.